

FlowGuard 技术说明

技术架构 · 工作流程 · 制作说明 | 确定性规则引擎 + AI 解释层 · 全站 TypeScript

一、技术架构

FlowGuard 采用「确定性规则引擎为决策核心 + AI 作为解释与补充信号层」的分层架构，全站 TypeScript，前后端同构于 Next.js App Router。

前端 / 应用层	Next.js 15 (App Router) + React + TypeScript + Tailwind CSS，移动端优先；react-i18next 中英双语。
服务端 / 接口层	Next.js Route Handlers (/app/api/*) 提供 REST 接口：付款、评估、放行、凭证、收款人、工单、通知等。
风控内核	确定性退回风险规则引擎 (src/lib/engine)，输出退回概率、命中因子、比价选路结果，可解释、可复现。
AI 能力层	Eazo App AI (DeepSeek) 经计费代理调用 (src/lib/eazo-ai-billing)，统一 /api/ai/insight 端点，多 kind 场景化提示词。
数据层	PostgreSQL + Drizzle ORM (postgres-js)；schema 覆盖 payments / suppliers / users / verification-cases。
集成与扩展	内置 MCP Server (/api/mcp) 对外暴露工具；ERP 投影、每日摘要通知 cron、公开凭证/案例分享链接。
工程质量	全站 TypeScript 0 error、中英双语键对齐、关键路由端到端可跑。

二、AI 的职责边界

规则引擎是唯一决策来源；AI
只把结构化快照转成自然语言洞察与建议，不发明数字、不改变评分。统一端点 /api/ai/insight
按场景分为多个 kind：

flow	说明资金当前卡在链路哪一节点、为何被卡、可以做什么。
route	解释规则引擎为何推荐这条通道、相较其他通道好在哪。
corridor	给出按国别 / 银行的结算要求清单。
reconcile	解释一处对账差异 / 未匹配凭证的原因。
risk-signals	仅由 AI 补充的语义 / 情境风险信号（只加警示、不降规则分）。

三、工作流程（一笔付款的完整链路）

- ① 发起 首页付款控制台查看待办与整体退回风险，点击新建付款进入向导。
- ② 退回预检 录入收款方后，规则引擎计算退回概率与命中因子，AI 补充语义风险并给出修复步骤。
- ③ 比价选路 在多个持牌通道间按费用 / 时效 / 退回率排序，AI 解释推荐理由，选定通道。
- ④ 审批放行 提交进入审批队列，Maker-Checker 双人复核，通过后生成付款指令。
- ⑤ 链路追踪 付款过程中逐跳追踪资金流经的中间行，标注透明度与滞留卡点。

⑥ 对账核销 到账后聚合进度与凭证，自动完成应收 vs 实收核销；历史页全程可回溯。

数据流：前端页面 → Route Handler (/api/payments、/assess、/review、/proof 等) → 规则引擎计算 → Drizzle 持久化到 PostgreSQL → 需要解释时调用 /api/ai/insight (DeepSeek) → 结果回渲染。

四、制作说明

技术选型	以确定性规则保证风控结果可解释、可复现；AI 只做解释层，规避“黑箱评分”与合规风险。
目录结构	src/app 路由与 API；src/lib/engine 规则引擎；src/lib/db Drizzle schema 与查询；src/components 页面与路演。
国际化	所有用户可见文案走 react-i18next，中英 (zh-CN / en-US) 双语键对齐，不硬编码。
合规落地	全站措辞统一为“纯软件决策辅助”，不持牌、不经手资金；结算由持牌机构完成。
路演材料	在线路演 /hackathon 与 PPTX 由同一份内容源 (hack-content.ts) 驱动，脚本一键导出，保持一致。
交付物	可运行 App、在线路演页、导出 PPTX、项目说明 PDF 与本技术说明 PDF。

在线演示：/hackathon | 路演材料：FlowGuard-Hackathon-Pitch.pptx | 产品说明：FlowGuard-项目说明.pdf